

AI ASSISTED CODING

LAB-13.2

NAME: D.siddhartha
ROLL NO:2503A51L04(B19)

Task Description #1 – Remove Repetition

Task: Provide AI with the following redundant code and ask it to refactor

Python Code

```
def calculate_area(shape, x, y=0):  
    if shape == "rectangle":  
        return x * y  
    elif shape == "square":  
        return x * x  
    elif shape == "circle":  
        return 3.14 * x * x
```

Expected Output

- Refactored version with dictionary-based dispatch or separate functions.
- Cleaner and modular design

PROMPT:

Python Code

```
def calculate_area(shape, x, y=0):  
    if shape == "rectangle":  
        return x * y  
    elif shape == "square":  
        return x * x  
    elif shape == "circle":  
        return 3.14 * x * x
```

for this code give Refactored version with dictionary-based dispatch or separate functions with Cleaner and modular design.

```

task1-13.py > calculate_area
1  def area_rectangle(x, y):
2      """Calculate area of a rectangle."""
3      return x * y
4
5  def area_square(x):
6      """Calculate area of a square."""
7      return x * x
8
9  def area_circle(x):
10     """Calculate area of a circle."""
11     return 3.14 * x * x
12
13  def calculate_area(shape, x, y=0):
14     """
15     Calculate area based on shape using dictionary-based dispatch.
16     """
17     dispatch = {
18         "rectangle": lambda: area_rectangle(x, y),
19         "square": lambda: area_square(x),
20         "circle": lambda: area_circle(x)
21     }
22     func = dispatch.get(shape.lower())
23     if func:
24         return func()
25     else:
26         raise ValueError("Unsupported shape")
27
28 # Example usage:
29 if __name__ == "__main__":
30     print("Rectangle area (5, 3):", calculate_area("rectangle", 5, 3))
31     print("Square area (4):", calculate_area("square", 4))
32     print("Circle area (3):", calculate_area("circle", 3))

```

OUTPUT:

```

Rectangle area (5, 3): 15
Square area (4): 16
Square area (4): 16
Circle area (3): 28.259999999999998

```

OBSERVATION:

The code was refactored from using multiple if-elif statements inside the calculate_area function to a cleaner, modular design. Now, each shape's area calculation is handled by a separate function, and a dictionary-based dispatch is used to select the correct function based on the shape.

Task Description #2 – Error Handling in Legacy Code

Task: Legacy function without proper error handling

Python Code

```
def read_file(filename):  
    f = open(filename, "r")  
    data = f.read()  
    f.close()  
    return data
```

Expected Output:

AI refactors with with open() and try-except

PROMPT:

```
def read_file(filename):  
    f = open(filename, "r")  
    data = f.read()  
    f.close()  
    return data
```

for the above code refactor with with open() and try-except let user give input to open and read the file.

```
TASK2-13.PY > ...  
1  def read_file(filename):  
2      """  
3      Reads the contents of a file safely using 'with open' and handles exceptions.  
4      """  
5      try:  
6          with open(filename, "r") as f:  
7              data = f.read()  
8              return data  
9      except FileNotFoundError:  
10         print(f"Error: The file '{filename}' was not found.")  
11         return None  
12     except Exception as e:  
13         print(f"An error occurred: {e}")  
14         return None  
15  
16  if __name__ == "__main__":  
17      filename = input("Enter the filename to open and read: ")  
18      content = read_file(filename)  
19      if content is not None:  
20          print("\nFile contents:\n")  
21          print(content)
```

OUTPUT:

```
Enter the filename to open and read: .sample.txt

File contents:

hi nagamrutha sahasra
uzma
ai assisted
hi
```

OBSERVATION:

The code safely reads and displays the contents of a user-specified file using with open and exception handling. It prompts the user for a filename, reads the file if it exists, and prints its contents. Errors such as missing files are handled gracefully with clear messages.

Task Description #3 – Complex Refactoring

Task: Provide this legacy class to AI for readability and modularity improvements:

Python Code

class Student:

def __init__(self, n, a, m1, m2, m3):

self.n = n

self.a = a

self.m1 = m1

self.m2 = m2

self.m3 = m3

def details(self):

print("Name:", self.n, "Age:", self.a)

def total(self):

return self.m1+self.m2+self.m3

Expected Output:

- AI improves naming (name, age, marks).
- Adds docstrings.
- Improves print readability.
- Possibly uses `sum(self.marks)` if marks stored in a list

PROMPT:

```
class Student:
def __init__(self, n, a, m1, m2, m3):
self.n = n
self.a = a
self.m1 = m1
self.m2 = m2
self.m3 = m3
def details(self):
print("Name:", self.n, "Age:", self.a)
def total(self):
return self.m1+self.m2+self.m3
```

for the above code improve naming (name, age, marks). Add docstrings and use `sum(self.marks)` if marks stored in a list.

```
task3-13.py > ...
1  class Student:
2      """
3      Represents a student with name, age, and marks in three subjects.
4      """
5
6      def __init__(self, name, age, mark1, mark2, mark3):
7          """
8          Initialize the student with name, age, and three marks.
9          Args:
10             name (str): Student's name.
11             age (int): Student's age.
12             mark1 (int/float): Mark in subject 1.
13             mark2 (int/float): Mark in subject 2.
14             mark3 (int/float): Mark in subject 3.
15         """
16         self.name = name
17         self.age = age
18         self.marks = [mark1, mark2, mark3]
19
20     def details(self):
21         """
22         Prints the student's name and age in a readable format.
23         """
24         print(f"Name: {self.name}, Age: {self.age}")
25
26     def total(self):
27         """
28         Returns the total marks obtained by the student.
29         """
30         return sum(self.marks)
31
32     # Example usage:
33     if __name__ == "__main__":
34         s = Student("Alice", 20, 85, 90, 95)
35         s.details()
36         print(f"Total Marks:", s.total())
```

OUTPUT:

```
Name: Alice, Age: 20  
Total Marks: 270
```

OBSERVATION:

The code now allows the user to input a student's name, age, and marks interactively. It stores the marks in a list for better structure and uses clear, descriptive variable names. The program prints the student's details and the total marks in a readable format, making it user-friendly and easy to maintain.

Task Description #4 – Inefficient Loop Refactoring

Task: Refactor this inefficient loop with AI help

Python Code

```
nums = [1,2,3,4,5,6,7,8,9,10]
```

```
squares = []
```

```
for i in nums:
```

```
    squares.append(i * i)
```

Expected Output: AI suggested a list comprehension

PROMPT:

Python Code

```
nums = [1,2,3,4,5,6,7,8,9,10]
```

```
squares = []
```

```
for i in nums:
```

```
    squares.append(i * i)
```

refactor the above code and suggest a list comprehension and print a example square

TASK4-13.PY > ...

```
1  nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
2  # Refactored using list comprehension
3  squares = [i * i for i in nums]
4  print(squares)
```

OUTPUT:

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

OBSERVATION:

The code efficiently generates a list of squares for numbers 1 to 10 using list comprehension, making it more concise and readable compared to a traditional loop. The output displays all the squared values in a single list.